Stationarity, Evaluation Cost, and Solution Quality in Nonconvex Unconstrained Optimization

Yangxiao Cai | Student ID: 26023041r
COMP6704 Individual Assignment, Fall 2026
Primary direction: Unconstrained Optimization

Abstract

We compare Armijo gradient descent (GD), safeguarded nonlinear conjugate gradient (CG), and BFGS on Rosenbrock and Rastrigin through 90 controlled CPU solves. Dimensions 2, 10, and 20 each use five paired initial points and fixed stopping criteria. In the recorded Windows reference run, 73/90 runs meet the gradient tolerance and 26/90 meet the global-objective tolerance: 47 meet only the gradient criterion, while 17 meet neither. On Rosenbrock, CG and BFGS attain the global-objective target in 14/15 and 12/15 cases; GD reaches the iteration cap in all 15 cases. On Rastrigin, 43/45 runs meet the gradient tolerance, but none meets the global-objective target. Endpoint diagnostics and comparisons conditional on CG and BFGS both meeting the global-objective target separate local convergence, evaluation cost, and final objective quality. Complete traces support this empirical distinction.

1 Introduction and scope

Smooth unconstrained optimization is a useful setting for studying how descent directions, line searches, curvature information, and stopping rules interact. It also exposes a practical ambiguity: software may report successful termination at a point with a small gradient even when its objective remains far from the global minimum. Comparing iteration counts alone can conceal both this difference in solution quality and the work spent rejecting trial steps.

This individual project belongs to Unconstrained Optimization. The variables range over all of real Euclidean space, and the objectives are nonconvex. The group project selected the Convex Optimization direction; the present investigation instead studies local algorithms outside convexity. Rosenbrock supplies a curved valley and coupled coordinates, while Rastrigin supplies many attraction basins and separable oscillations. Both have known global objective value zero, making stationarity and global-objective quality independently measurable [7, 8].

Three questions guide the comparison. First, which methods reach the common gradient tolerance within a fixed accepted-update budget? Second, what function and gradient evaluation costs accompany their outcomes across dimensions and paired starts? Third, when termination is unsuccessful or non-global, what do the saved traces and endpoint curvature actually support? The design fixes the implementations and settings before comparing all 90 outcomes, including failures.

Code and data: `https://github.com/Caiyangxiao/COMP6704-Individual-CYX`

The repository's README.md provides setup, verification, and reproduction commands. The PDF also embeds an offline snapshot as `COMP6704_reproducibility.zip`.

2 Literature review and connection to the course

Classical foundation. Nocedal and Wright [1] organize smooth unconstrained methods around a search direction and an inexact line search. Their treatment connects first-order stationarity, descent, quasi-Newton curvature, and local convergence. These concepts provide the classroom framework here: an update can decrease the objective while remaining slow, and a necessary first-order condition becomes sufficient for global optimality only under additional structure such as convexity. Our nonconvex functions deliberately remove that sufficiency.

Step selection and memory. Armijo's early work [2] established a sufficient-decrease approach to choosing step lengths. GD uses the current negative gradient, making its behavior easy to audit but sensitive to geometry. Nonlinear CG carries information from the preceding search direction. Gilbert and Nocedal [3] analyze globally convergent CG variants with appropriate safeguards and line searches on smooth nonconvex problems. Such results depend on the precise variant and assumptions; they do not imply that an arbitrary Polak–Ribiere implementation or a positive β alone ensures descent. The PR+ implementation used here therefore warrants source-level identification.

Quasi-Newton methods. Fletcher's variable-metric development [4] is part of the foundation of BFGS. BFGS learns curvature from successive displacements and gradient differences instead of forming an analytic Hessian. This can improve local progress, at the cost of matrix storage and updates. Positive-definite inverse-Hessian approximations yield descent directions in exact arithmetic, but neither this property nor a Wolfe line search certifies a global minimizer on a nonconvex objective. Implementation details and finite precision remain relevant when a solver stops near its tolerance.

Recent theory. Jin, Jiang, and Mokhtari [5] obtain explicit non-asymptotic rates for BFGS with Armijo–Wolfe line search under strong convexity and smoothness; additional regularity supports stronger rate statements. Their analysis clarifies the gap between qualitative convergence and finite-iteration performance. Strong convexity is absent globally in our tests, so these rates provide context rather than a theorem for the recorded runs. Mannel [6] studies a modified, globalized L-BFGS and Barzilai–Borwein framework for nonconvex unconstrained optimization, using cautious updates and conditions supporting stationarity of cluster points. This is a different algorithm from ordinary full-memory SciPy BFGS. Its relevance is that nonconvex guarantees require carefully designed safeguards, rather than a change of terminology.

Source / line of work	**Relevance to this study**	**Boundary of the comparison**
Armijo [2]; textbook [1]	Sufficient decrease; descent-lemma reasoning	Finite backtracking caps can terminate early
Gilbert–Nocedal [3]	Nonlinear CG safeguards and nonconvex analysis	Identify the actual PR+ implementation
Jin et al. [5], 2024	Finite-time BFGS analysis with inexact line search	Strong-convexity rates do not apply globally here
Mannel [6], 2025	Nonconvex globalization of limited-memory methods	Modified L-BFGS is not the tested full BFGS

Study position. The controlled comparison distinguishes satisfaction of the gradient criterion, objective quality, and evaluation cost using complete traces. Version-pinned SciPy documentation and source [7] determine the implemented CG and BFGS behavior. Known optima and transparent benchmark geometry make discrepancies between success labels and objective quality directly measurable; Section 9 discusses the limits of this design.

3 Formal problem and objective geometry

We solve $\min f(x)$ over $x \in \mathbb{R}^d$, with d in {2, 10, 20}; no feasibility constraints or projections are imposed. Write $g_k = \nabla f(x_k)$, use the Euclidean `norm` throughout, and denote the known global objective by $f^\star = 0$. The two benchmark models are

$$f_R(x) = \sum_{i=1}^{d-1} \left[100(x_{i+1} - x_i^2)^2 + (1 - x_i)^2\right] \tag{1}$$

$$f_A(x) = \sum_{i=1}^{d} \left[x_i^2 + 20\sin^2(\pi x_i)\right] \tag{2}$$

The Rastrigin expression is algebraically equivalent to $10d + \sum_i [x_i^2 - 10\cos(2\pi x_i)]$ [8]. The sine-squared form avoids cancellation between large constants near zero. Rosenbrock uses SciPy's `rosen` and `rosen_der`; Rastrigin has the analytic derivative

$$[\nabla f_A(x)]_i = 2x_i + 20\pi \sin(2\pi x_i) \tag{3}$$

For Rosenbrock, each term indexed by i contributes $-400x_i(x_{i+1} - x_i^2) + 2(x_i - 1)$ to derivative component i and $200(x_{i+1} - x_i^2)$ to component i+1. The implementation accumulates all adjacent contributions. Before solving, central differences at 12 nonstationary test points check both gradients, using $h_i = 10^{-6}\max(1, |x_i|)$ and componentwise absolute and relative tolerances 10^{-5}. All 12 checks pass; those checks are outside solver evaluation counts.

Known optima and nonconvexity. Both models are sums of nonnegative terms. Rosenbrock reaches zero only at the all-ones vector for d $\geq$ 2; Rastrigin reaches zero only at the zero vector. In two-dimensional Rosenbrock, the Hessian at (0,1) is diag(-398,200), so the function is not convex. This point is not a saddle point: its gradient is (-2,200), which is nonzero. For Rastrigin the Hessian is diagonal with entries $2 + 40\pi^2\cos(2\pi x_i)$; a coordinate equal to 1/2 yields negative curvature. Negative curvature at an arbitrary point establishes nonconvexity, not stationarity.

The coupled Rosenbrock valley makes curvature vary strongly with direction. Rastrigin is separable, yet its oscillatory terms produce many local basins; coordinatewise independence therefore does not make a local descent method a global solver. Both functions are smooth and coercive, so bounded objective sublevels are compact. Their gradients have suitable local Lipschitz bounds on bounded regions, but Rosenbrock's gradient is not globally Lipschitz over all of $\mathbb{R}^d$. At the two-dimensional Rosenbrock global minimizer, the Hessian has eigenvalues approximately 0.39936 and 1001.60064, giving condition number about 2508. This local ill-conditioning is consistent with slow GD progress; it is not a rate estimate for the full nonlinear trajectory. These distinctions matter when stating descent guarantees.

4 Algorithms and implementation

4.1 Gradient descent with Armijo backtracking

$$\begin{aligned} g_k = \nabla f(x_k), \qquad & x_{k+1} = x_k - \alpha_k g_k \\ f(x_k - \alpha_k g_k) \leq f(x_k) - c_1\alpha_k \|g_k\|_2^2 & \end{aligned} \tag{4}$$

GD is implemented directly. Every iteration starts again at α_0 = 1, halves the trial step with τ = 0.5, and accepts the first point satisfying (4), with c_1 = 10^{-4}. At most 50 reductions follow the initial trial, and the minimum step is 10^{-12}. A rejected trial never updates x. Nonfinite values, an unchanged floating-point point, and an exhausted line search receive explicit failure statuses. The accepted-update limit is 5000; all function calls at rejected trial points still count.

4.2 Safeguarded nonlinear conjugate gradient

$$p_0 = -g_0, \qquad p_k = -g_k + \beta_k^{\mathrm{PR+}} p_{k-1} \quad (k \geq 1)$$
$$\beta_k^{\mathrm{PR+}} = \max\left\{0, \frac{g_k^\top (g_k - g_{k-1})}{g_{k-1}^\top g_{k-1}}\right\} \tag{5}$$

CG uses `scipy.optimize.minimize(method='CG')` with analytic gradients. Equation (5) is the PR+ direction recurrence, initialized with $p_0 = -g_0$ [3, 7]. SciPy 1.13.1 adds a sufficient-descent acceptance check for the proposed next direction: the inner product of the next direction and gradient must not exceed -0.01 times the squared gradient `norm`, unless the gradient target is already met. PR+ truncation alone does not guarantee this property. This safeguard is not a fixed periodic restart. Both SciPy methods use the strong-Wolfe conditions

$$f(x_{k+1}) \leq f(x_k) + c_1 \alpha_k g_k^\top p_k$$
$$|g_{k+1}^\top p_k| \leq c_2 |g_k^\top p_k|, \qquad 0 < c_1 < c_2 < 1 \tag{6}$$

Here $0 < c_1 = 10^{-4} < c_2 = 0.4 < 1$. The same constants reduce one source of variation between CG and BFGS, although their search directions and internal trial sequences still differ. Function and gradient counts consequently remain necessary measurements.

4.3 Full-memory BFGS

BFGS uses `scipy.optimize.minimize(method='BFGS')`, $p_k = -H_k g_k$, and $H_0 = I$. With displacement s_k and gradient change y_k as defined below, the inverse-Hessian update is [1, 4]

$$H_{k+1} = (I - \rho_k s_k y_k^\top) H_k (I - \rho_k y_k s_k^\top) + \rho_k s_k s_k^\top$$
$$s_k = x_{k+1} - x_k, \quad y_k = g_{k+1} - g_k, \quad \rho_k = (y_k^\top s_k)^{-1} \tag{7}$$

4.4 Ideal Armijo GD stationarity bound

For GD, assume f is bounded below by $f_{\inf}$ and its gradient is L-Lipschitz with one uniform constant L on all relevant trial segments. Coercivity bounds accepted iterates; bounded gradients there and trial steps no larger than one contain the trial segments in a compact region, where smoothness supplies such an L. The descent lemma makes any $\alpha \leq 2(1 - c_1)/L$ sufficient for (4). With ideal unlimited backtracking from one, an accepted α is at least the lower bound below. Summing (4) over K updates gives

$$\alpha_{\mathrm{low}} = \min\left\{1, \frac{2\tau(1 - c_1)}{L}\right\} > 0$$
$$\min_{0 \leq k < K} \|g_k\|_2^2 \leq \frac{f(x_0) - f_{\inf}}{c_1 \alpha_{\mathrm{low}} K} \tag{8}$$

Thus ideal Armijo GD has a standard $\mathcal{O}(\epsilon^{-2})$ accepted-update bound for finding an iterate with gradient `norm` at most ϵ under these assumptions. This is a stationarity result, not a global-optimality result. The actual finite iteration, trial, and minimum-step limits can stop a run before it reaches that target; the asymptotic statement does not override its recorded failure.

4.5 BFGS curvature implication

For a descent direction p_k and an accepted strong-Wolfe step, (6) gives the curvature inequality

$$s_k^\top y_k \geq \alpha_k (1 - c_2)(-g_k^\top p_k) > 0 \tag{9}$$

In exact arithmetic, (7) therefore preserves positive definiteness from H_0=I, even when the true Hessian is indefinite. This curvature property does not ensure global optimality or prevent finite-precision failure. The strong-convexity rates in [5] do not apply globally here; CG convergence likewise requires its line-search and regularity hypotheses.

5 Experimental protocol, costs, and auditing

Component	Fixed protocol
Design	2 functions x 3 dimensions x 5 paired starts x 3 methods = 90 solves
Initialization	Seed 20261105; `PCG64` with `SeedSequence`([seed, objective_index, d]); coordinates uniform on [-2,2] for Rosenbrock and [-3,3] for Rastrigin
Stopping	Euclidean gradient `norm` $\leq 10^{-6}$; at most 5000 accepted updates
SciPy options	CG/BFGS: c_1=10^{-4}, c_2=0.4, `norm`=2, `gtol`=10^{-6}, `maxiter`=5000. BFGS: $H_0 = I$, `xrtol`=0; default c_2=0.9 is overridden. No analytic Hessian or damping.
Arithmetic / environment	`float64`; Python 3.12.8, NumPy 1.26.4, SciPy 1.13.1; Windows 11; sequential CPU solves; BLAS/OpenMP thread variables set to 1
Hardware record	Intel64 Family 6 Model 191 Stepping 2; 24 logical CPUs reported; no GPU used
Independent outcomes	Native solver status; S: finite result and $\|\nabla f\|_2 \leq 10^{-6}$; G: finite result and $\|f - f^\star\| \leq 10^{-6}$

The intervals above generate initial points only; they do not constrain subsequent iterates. Each function-dimension pair has its own random stream, and each of its five saved starts is shared across methods. This pairing controls starting difficulty without asserting that five points represent all possible initializations. The two tolerances define different questions. Native `solver_success` and S happen to agree for these 90 runs, but remain distinct fields; G is a known-objective target rather than a mathematical certificate of exact optimality.

Evaluation accounting. A wrapper counts every actual raw objective and gradient call made by a solver, including rejected line-search trials and trials in a failed terminal search. Calls avoided by SciPy's cache are not invented. The wrapper counts must equal SciPy's returned `nfev`/`njev`. Callbacks only copy accepted coordinates and snapshot counters; they do not reevaluate f or g. Each trace contains an initial event with counters zero, one event per accepted update, and a terminal event. Initial plot diagnostics and every recorded objective/gradient `norm` are filled offline from raw functions, outside the counters and timer. Thus the initial point can appear at `nfev`=0 without making the solver's own initial evaluation free.

The study records 80,605 events: 80,425 accepted updates and 180 initial/terminal events. Solver time covers evaluations, updates, and event capture, excluding imports, offline diagnostics, file I/O, and plotting. The sum of measured solver times is 26.915 s, rather than total pipeline wall time. Timing is secondary because these small problems are sensitive to Python, callback, and scheduling overhead. Figures stop at each actual endpoint; no curve is extended after termination, and log-display floors never change raw values.

Method	Direction / update arithmetic	Solver state
GD	$\mathcal{O}(d)$ per direction; line-search calls additional	$\mathcal{O}(d)$
PR+ CG	$\mathcal{O}(d)$ per direction; line-search calls additional	$\mathcal{O}(d)$
BFGS	$\mathcal{O}(d^2)$ direction; standard low-rank update $\mathcal{O}(d^2)$; tested SciPy dense-product update $\mathcal{O}(d^3)$*	$\mathcal{O}(d^2)$

*Table 1. Algorithmic work excludes objective/gradient calls, each $\mathcal{O}(d)$ for these models. *SciPy 1.13.1 forms A1 @ H @ A2 using generic dense products [7]; $\mathcal{O}(d^3)$ uses conventional dense arithmetic, not an empirical scaling fit. Storing K d-dimensional trace points adds $\mathcal{O}(Kd)$ space for every method.*

Audit checks cover pairings, trace structure, monotone counters, GD sufficient decrease, endpoints, native counts, and source hashes. All 50 unit tests and the independent full-study validator pass.

6 Results: Rosenbrock

d	Method	S/G	Updates median	nfev median [range]	njev median [range]	Final f median [range]
2	GD	0/0	5000	49290 [49090, 51127]	5001 [5001, 5001]	4.64×10^{-6} [3.35×10^{-6}, 5.07×10^{-5}]
2	CG	5/5	23	56 [32, 72]	56 [32, 72]	2.49×10^{-16} [4.24×10^{-20}, 3.03×10^{-13}]
2	BFGS	5/5	25	48 [20, 58]	48 [20, 58]	1.33×10^{-17} [8.99×10^{-21}, 8.49×10^{-15}]
10	GD	0/0	5000	53100 [53063, 53339]	5001 [5001, 5001]	4.91×10^{-4} [4.14×10^{-4}, 4.99×10^{-4}]
10	CG	5/4	249	434 [282, 553]	434 [282, 553]	9.47×10^{-14} [2.11×10^{-14}, 3.99]
10	BFGS	5/3	70	114 [100, 146]	114 [100, 146]	3.08×10^{-17} [2.80×10^{-18}, 3.99]
20	GD	0/0	5000	53289 [53071, 53376]	5001 [5001, 5001]	0.00353 [0.00339, 0.004]
20	CG	5/5	444	709 [482, 900]	709 [482, 900]	1.10×10^{-13} [5.87×10^{-16}, 1.35×10^{-13}]
20	BFGS	5/4	110	175 [166, 203]	175 [166, 203]	3.72×10^{-16} [4.74×10^{-18}, 3.99]

Table 2. Rosenbrock, five starts in every row. S and G are counts out of five, not a ratio. Ranges are minima and maxima; failed runs remain included. Complete update and time ranges are in `grouped_results.csv`. Scientific notation is rounded only for display.

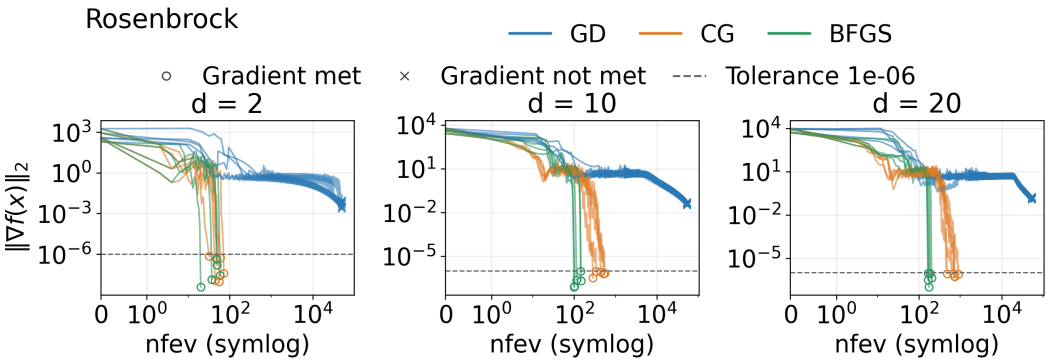


Figure 1. Gradient `norm` against cumulative function evaluations for all five starts in each dimension. Colors denote methods; circles meet S and crosses do not. The x-axis is `symlog` with a linear region at zero; each curve stops at termination. The horizontal dashed line is 10^{-6}.

Budget and convergence. CG and BFGS meet the gradient criterion in all 15 Rosenbrock runs each; GD meets it in none. Every GD run accepts all 5000 permitted updates, with no failed line search. The final gradient medians are 5.31×10^{-3}, 4.30×10^{-2}, and 1.26×10^{-1} at d=2,10,20, well above tolerance. Accepted-step medians are 1/512, 1/1024, and 1/1024, while median trial counts per accepted step are 10, 11, and 11. Their large function counts reflect repeated trial rejection as well as slow progress in the valley. The geometry and reset-to-one policy offer a plausible explanation; this study has no ablation to isolate their causal contributions.

7 Results: Rastrigin

d	Method	S/G	Updates median	nfev median [range]	njev median [range]	Final f median [range]
2	GD	5/0	31	276 [249, 313]	32 [29, 37]	4.97 [0.995, 12.9]
2	CG	5/0	7	19 [10, 24]	19 [10, 24]	9.95 [3.98, 16.9]
2	BFGS	5/0	6	12 [9, 16]	12 [9, 16]	9.95 [3.98, 16.9]
10	GD	5/0	34	305 [288, 312]	35 [33, 36]	35.8 [25.9, 37.8]
10	CG	5/0	9	19 [14, 24]	19 [14, 24]	27.9 [23.9, 37.8]
10	BFGS	5/0	20	39 [32, 43]	39 [32, 43]	27.9 [23.9, 37.8]
20	GD	5/0	34	305 [293, 312]	35 [34, 36]	64.7 [41.8, 126]
20	CG	5/0	9	19 [15, 22]	19 [15, 22]	63.7 [41.8, 69.6]
20	BFGS	3/0	26	86 [45, 117]	86 [45, 105]	62.7 [41.8, 68.7]

Table 3. Rastrigin uses the same definitions and all-run convention as Table 2. In the final row, `nfev` and `njev` have equal medians but different ranges because one terminal search adds unequal function and gradient counts.

Stationarity and objective quality separate sharply. GD and CG each meet S in 15/15 runs; BFGS meets S in 13/15. None of the 45 runs meets G. Hence 43 runs meet the gradient criterion while missing the global-objective target. Across each dimension, CG uses fewer function evaluations than GD: medians 19,19,19 versus 276,305,305. BFGS's corresponding medians are 12,39,86. These counts describe function evaluations to the observed endpoints; differing final objective values prevent an equal-quality cost comparison.

The paired final objectives in Figure 2 show how rankings depend on the outcome. At d=2, GD obtains a lower f than both other methods on four of the five starts, despite using more function evaluations. On start 1 it returns f=0.994959 with 313 function evaluations, compared with CG's 4.974790 using 24 and BFGS's 7.959662 using 10. All three methods meet the gradient criterion. For this starting point, GD achieves the lowest final objective value but requires the most function evaluations. Raw Rastrigin objectives add contributions across coordinates; their increase across dimensions alone is not evidence of worsening per-coordinate quality.

7.1 The two BFGS precision-loss terminations

20D start	Updates	nfev / njev	Final gradient norm	Final f
3	26	117 / 105	8.73821×10^{-6}	41.78821486
4	21	45 / 45	$1.00023457 \times 10^{-6}$	64.67220759

Both retain SciPy status=2 and its precision-loss message. At start 3 the last accepted event has counts 59/59; the terminal event adds 58 function and 46 gradient evaluations without accepting a new point. This directly exposes unsuccessful terminal trial work that iteration counts omit. At start 4 there are no extra terminal calls, and the last accepted objective change rounds to zero. Its final gradient `norm` is only 0.02346% above the threshold, but it remains a failure under the fixed rule. Relaxing the threshold after seeing the result would change the protocol.

The common message does not uniquely identify an internal mechanism for each run. Failed line-search progress and finite-precision resolution are consistent with the saved evidence, but the traces do not expose every internal scalar. The analytic derivations and independent checks provide no evidence of a gradient implementation error. Finite checks do not exclude every numerical problem. Both outcomes are included in the final row of Table 3 and shown as crosses in Figure 2.

8 Paired outcomes and endpoint interpretation

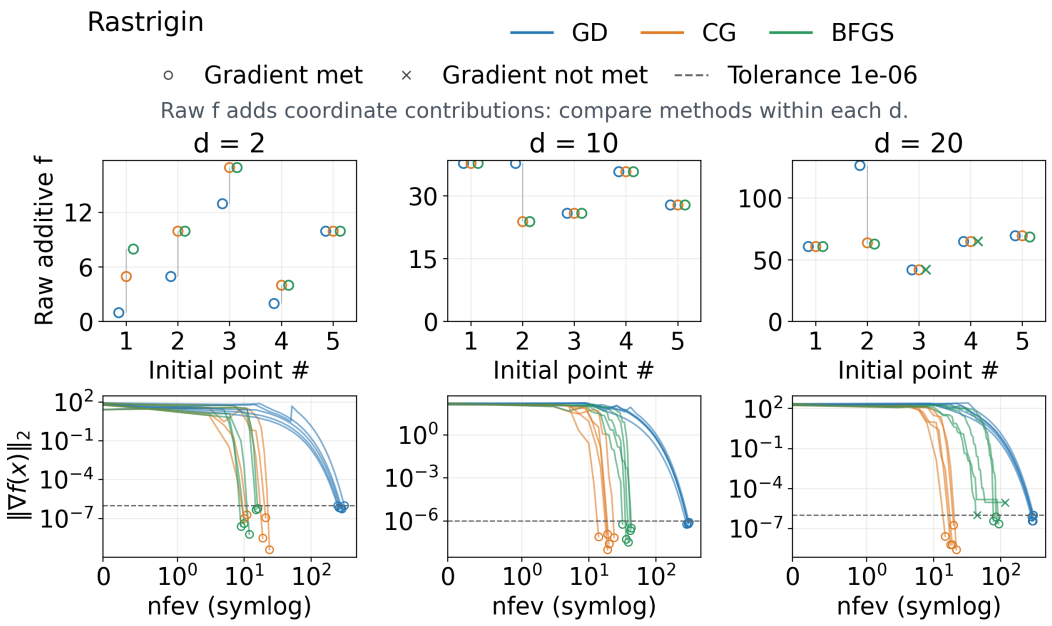


Figure 2. All Rastrigin starts. Top: final raw f, with faint vertical segments connecting the range of three outcomes for each shared start. Bottom: gradient `norm` against cumulative `nfev`, with `symlog` x and log y. Colors denote methods; circles meet S and crosses do not. Compare objective quality within each dimension because f is additive. Curves end at their true termination, including unsuccessful runs.

8.1 Non-global Rosenbrock endpoints

d / start / method	Final f	Gradient `norm`	Minimum Hessian eigenvalue
10 / 4 / CG	3.9865791123	9.56589×10^{-7}	0.501273
10 / 4 / BFGS	3.9865791123	9.63817×10^{-7}	0.501273
10 / 5 / BFGS	3.9865791123	8.78814×10^{-8}	0.501273
20 / 1 / BFGS	3.9866238543	4.44622×10^{-7}	0.498755

Table 4. The four Rosenbrock endpoints meeting S but missing G. Hessian spectra are computed offline and excluded from solve time and evaluation counts.

These four endpoints explain why 30 Rosenbrock runs meet the gradient criterion but only 26 meet the global-objective target. Positive numerical Hessian eigenvalues together with small gradients support the interpretation that the solvers finish near non-global local minima. At an approximate point, those quantities are numerical evidence, not an exact proof of a stationary local minimizer. Several methods can therefore converge convincingly while legitimately differing from the known global optimum. The contrast with GD's 15 budget-exhausted runs also separates convergence to a different basin from failure to reach stationarity within the budget.

Evaluation cost at comparable quality. BFGS's `nfev` medians rise from 48 to 114 to 175, while CG's rise from 56 to 434 to 709. Different counts of runs meeting G prevent an equal-quality comparison of these unconditional medians. Restricting to shared initial points for which both methods meet G leaves 5, 3, and 4 pairs for d=2,10,20; the median ratios of function evaluations, `nfev`(CG)/`nfev`(BFGS), are 1.24, 2.97, and 3.91. These median ratios favor BFGS in function evaluations; they do not measure wall time or describe starts where either method misses G. CG meets G in 14/15 runs overall versus BFGS in 12/15.

9 Discussion, limitations, and reproducibility

Interpretation of the three questions. First, in the recorded Windows reference run, 73/90 runs meet the common gradient criterion, with failures concentrated in Rosenbrock GD and two 20D Rastrigin BFGS runs. Second, function and gradient counts reveal costs hidden by accepted-update counts; the terminal 58/46 extra calls are a concrete example. The median ratios favor BFGS in function evaluations on the selected Rosenbrock pairs where both methods meet G, whereas CG uses fewer function evaluations than GD to its observed Rastrigin stationary endpoints. Third, final f and endpoint diagnostics show that meeting the gradient criterion does not imply meeting the global-objective target: 47 runs meet S while missing G. These observations connect descent and curvature concepts from the course to actual solver behavior.

Limits of the evidence. The study includes only two synthetic functions, three dimensions up to 20, and five fixed starts per setting. Its observed fractions are not estimated population success probabilities. Dimensions and changing random starts vary together, so trends across dimensions are descriptive. The experiment neither establishes asymptotic scaling nor measures the practical memory limit of BFGS. A single implementation and one set of line-search constants also cannot isolate the effects of direction, step selection, initialization, tolerance, and numerical safeguards. Millisecond timings do not justify a universal fastest-method claim, and conditioning cost ratios on both methods attaining G creates an explicitly selected subset.

Platform scope. The tables describe the recorded Windows run. Pinning NumPy and SciPy versions does not guarantee identical floating-point paths across operating systems, Python builds, processors, or linked numerical libraries. Small arithmetic differences can change line-search decisions, evaluation counts, and classifications near the strict 10^{-6} threshold. A different execution path does not, by itself, invalidate the saved Windows records. Such differences require trace-level diagnosis; this study does not isolate a platform-specific cause.

Useful follow-up work would vary GD's initial trial-step policy under the same starts, sweep the stopping tolerance and iteration budget, and compare damped or limited-memory quasi-Newton variants using a larger test collection. Multi-start global search would address basin selection but would answer a different question from this fixed local-method comparison. A real-data smooth objective would improve external relevance while usually removing access to a known global optimum. These are future experiments, not results claimed here.

9.1 Reproducibility and access

Start with README.md in the GitHub repository linked on page 1. The PDF also embeds an offline snapshot as `COMP6704_reproducibility.zip`, with `packaging_verification.json` attached separately. Historical Windows-to-Windows comparisons, archived unit tests, and read-only data/source checks are dated and scoped separately; they establish neither a new formal experiment nor cross-platform numerical identity.

10 Conclusion

Under the fixed protocol, convergence to a small-gradient point did not imply a small global-objective gap. Rosenbrock favored CG and BFGS over this GD configuration within the iteration budget, but both sometimes reached non-global near-stationary points. On Rastrigin, all global-objective targets were missed despite 43/45 runs meeting the gradient criterion; GD occasionally obtained better objective values while using more function evaluations. The two BFGS precision-loss terminations were retained, including one just above tolerance. These results support separate reporting of stationarity, objective quality, costs, and failure modes.

References

[1] J. Nocedal and S. J. Wright. Numerical Optimization, 2nd ed. Springer, 2006. DOI: 10.1007/978-0-387-40065-5. https://link.springer.com/book/10.1007/978-0-387-40065-5

[2] L. Armijo. Minimization of functions having Lipschitz continuous first partial derivatives. Pacific Journal of Mathematics, 16(1):1–3, 1966. https://msp.org/pjm/1966/16-1/pjm-v16-n1-p01-p.pdf

[3] J. C. Gilbert and J. Nocedal. Global convergence properties of conjugate gradient methods for optimization. SIAM Journal on Optimization, 2(1):21–42, 1992. DOI: 10.1137/0802003. https://doi.org/10.1137/0802003

[4] R. Fletcher. A new approach to variable metric algorithms. The Computer Journal, 13(3):317–322, 1970. DOI: 10.1093/comjnl/13.3.317. https://doi.org/10.1093/comjnl/13.3.317

[5] Q. Jin, R. Jiang, and A. Mokhtari. Non-asymptotic global convergence analysis of BFGS with the Armijo-Wolfe line search. Advances in Neural Information Processing Systems 37:16810–16851, 2024. DOI: 10.52202/079017-0536. Author version: arXiv:2404.16731, revised January 8, 2025. https://arxiv.org/abs/2404.16731

[6] F. Mannel. A globalization of L-BFGS and the Barzilai–Borwein method for nonconvex unconstrained optimization. Journal of Optimization Theory and Applications, 204, Article 47, 2025. DOI: 10.1007/s10957-024-02565-5. https://link.springer.com/article/10.1007/s10957-024-02565-5

[7] SciPy developers. SciPy 1.13.1: minimize-CG and minimize-BFGS reference documentation and optimization tutorial; scipy/optimize/_optimize.py, tag v1.13.1. Version-specific implementation and stopping/line-search options. Accessed November 5, 2026. https://github.com/scipy/scipy/blob/v1.13.1/scipy/optimize/_optimize.py

[8] S. Surjanovic and D. Bingham. Virtual Library of Simulation Experiments: Test Functions and Datasets—Rastrigin Function. Simon Fraser University. Accessed November 5, 2026. https://www.sfu.ca/~ssurjano/rastr.html

Version-specific option references supporting [7]:

CG documentation:
https://docs.scipy.org/doc/scipy-1.13.1/reference/optimize.minimize-cg.html

BFGS documentation:
https://docs.scipy.org/doc/scipy-1.13.1/reference/optimize.minimize-bfgs.html

The course page limit includes this reference list. The PDF uses the official NeurIPS 2025 style with a course-specific notice.